LLM ENGINEER INTERVIEW QUESTIONS

🚀 100 MOST IMPORTANT LLM & GENERATIVE AI INTERVIEW Q&A

(Easy to revise + Interview Ready)

☑ SECTION 1 — Generative AI Basics (1–15)

1. What is Generative AI?

Generative AI is a type of AI that creates new content such as text, images, audio, or code by learning patterns from training data instead of only making predictions.

2. What is an LLM?

LLM (Large Language Model) is a deep learning model trained on massive text datasets using transformer architecture to understand and generate human-like language.

3. Difference between AI, ML, DL, and Generative AI?

- AI → intelligent systems
- ML → learns from data
- DL → neural networks
- Generative AI → creates new content

4. Examples of LLMs?

GPT, LLaMA, Claude, Gemini, Mistral.

5. What tasks can LLMs perform?

Text generation, summarization, translation, Q&A, coding, reasoning, chatbots.

6. What makes LLMs powerful?

Large datasets + transformer architecture + self-attention + scaling.

7. What is token in LLM?

A token is a small unit of text (word/subword/character) processed by the model.

8. What is tokenization?

Process of converting text into tokens before feeding into model.

9. What is context window?

Maximum number of tokens model can remember in one request.

10. What is prompt?

Input instruction given to an LLM to generate output.

11. What is zero-shot learning?

Model performs task without examples.

12. Few-shot learning?

Providing few examples inside prompt.

13. One-shot learning?

Only one example provided.

14. What is hallucination?

When LLM generates incorrect but confident answers.

15. Why do hallucinations happen?

Training data uncertainty + probability-based prediction + lack of real-world verification.

☑ SECTION 2 — Transformer & Architecture (16–30)

16. What architecture do LLMs use?

Transformer architecture.

17. Who introduced Transformers?

Vaswani et al., 2017 — *Attention Is All You Need*.

18. What is Attention Mechanism?

It allows model to focus on important words while processing sentences.

19. What is Self-Attention?

Each token attends to all other tokens in the sequence.

20. Why transformers replaced RNNs?

Parallel computation + long-range dependency handling.

21. Components of Transformer?

Encoder, Decoder, Attention, Feedforward layers.

22. What is positional encoding?

Adds word order information since transformers process tokens simultaneously.

23. Multi-head attention?

Multiple attention layers learn different relationships simultaneously.

24. Encoder vs Decoder?

Encoder → understand input

Decoder → generate output

25. GPT uses encoder or decoder?

Decoder-only architecture.

26. BERT architecture?

Encoder-only.

27. Why attention is important?

Captures context relationships between words.

28. What is embedding?

Numerical vector representation of words.

29. What is embedding dimension?

Size of vector representing tokens.

30. What happens during forward pass?

Tokens → embeddings → attention → prediction probabilities.

☑ SECTION 3 — Training LLMs (31–45)

31. How are LLMs trained?

Pretraining + Fine-tuning.

32. What is pretraining?

Training on massive unlabeled text using next-token prediction.

33. Objective of GPT training?

Predict next token.

34. What is fine-tuning?

Training model on task-specific dataset.

35. Instruction tuning?

Training model to follow human instructions.

36. What is RLHF?

Reinforcement Learning from Human Feedback improves response quality using human preferences.

37. Steps in RLHF?

Supervised fine-tuning → reward model → reinforcement optimization.

38. What is reward model?

Model scoring output quality.

39. Why RLHF used?

Align model behavior with humans.

40. What is dataset curation?

Cleaning and selecting training data.

41. What is overfitting in LLM?

Model memorizes instead of generalizing.

42. What is scaling law?

Performance improves with more data, parameters, and compute.

43. What are parameters?

Learnable weights inside model.

44. What is temperature?

Controls randomness of output.

45. Top-p sampling?

Select tokens from cumulative probability distribution.

☑ SECTION 4 — Prompt Engineering (46–60)

46. What is Prompt Engineering?

Designing prompts to get better outputs.

47. Types of prompts?

Zero-shot, Few-shot, Chain-of-thought.

48. Chain-of-Thought prompting?

Model explains reasoning step-by-step.

49. System prompt?

Defines assistant behavior.

50. Role prompting?

Assign role like “Act as doctor”.

51. Why prompt matters?

LLMs heavily depend on input structure.

52. What is prompt injection?

Malicious input manipulating model behavior.

53. How prevent prompt injection?

Input validation + guardrails.

54. Output formatting prompts?

Force JSON or structured outputs.

55. Temperature vs Top-p?

Temperature controls randomness globally; Top-p limits probability mass.

56. Best prompt design rule?

Clear instruction + context + format.

57. What is few-shot advantage?

Improves accuracy without retraining.

58. Delimiter usage?

Separates instructions and context.

59. Prompt evaluation?

Testing prompts for consistency and accuracy.

60. Prompt templates?

Reusable structured prompts.

☑ SECTION 5 — RAG (Retrieval Augmented Generation) (61–75)

61. What is RAG?

Combines retrieval system with LLM to answer using external knowledge.

62. Why RAG used?

Reduces hallucinations and adds real-time knowledge.

63. RAG pipeline steps?

Chunk → Embed → Store → Retrieve → Generate.

64. What is vector database?

Stores embeddings for similarity search.

65. Examples of vector DB?

Pinecone, FAISS, Chroma, Weaviate.

66. What is embedding model?

Converts text into vectors.

67. Similarity search?

Find closest vectors using cosine similarity.

68. Chunking?

Splitting documents into smaller parts.

69. Why chunking important?

Fits context window and improves retrieval.

70. Metadata filtering?

Search documents using tags.

71. Hybrid search?

Keyword + vector search.

72. Retriever vs Generator?

Retriever fetches info; Generator produces answer.

73. Re-ranking?

Sorting retrieved documents by relevance.

74. Context injection?

Adding retrieved text into prompt.

75. RAG limitation?

Depends on retrieval quality.

☑ SECTION 6 — LLM Engineering & Deployment (76–90)

76. What is LLMOps?

Operational lifecycle of LLM applications.

77. Latency in LLM apps?

Time taken to generate response.

78. How reduce latency?

Caching, smaller models, batching.

79. Quantization?

Reducing precision to make models faster.

80. LoRA?

Parameter-efficient fine-tuning method.

81. Why LoRA used?

Fine-tune large models cheaply.

82. Model distillation?

Train small model from large model outputs.

83. API vs Self-hosted LLM?

API easier; self-hosted gives control.

84. Streaming response?

Tokens sent gradually.

85. Guardrails?

Safety constraints on output.

86. Evaluation metrics?

BLEU, ROUGE, Human evaluation.

87. Hallucination detection?

Grounding + verification systems.

88. Caching in LLM apps?

Store repeated responses.

89. Rate limiting?

Control API usage.

90. Monitoring LLM apps?

Track latency, cost, accuracy.

☑ SECTION 7 — Advanced Interview Topics (91–100)

91. What are AI Agents?

LLMs that take actions using tools.

92. Tool calling?

LLM invokes APIs/functions.

93. Function calling?

Structured API interaction.

94. Agent framework examples?

LangChain, LangGraph, CrewAI.

95. Memory in agents?

Stores conversation history.

96. Short-term vs Long-term memory?

Session vs persistent storage.

97. Multi-agent system?

Multiple AI agents collaborate.

98. Evaluation of agents?

Task success + reasoning accuracy.

99. Biggest challenge in LLM engineering?

Hallucination, cost, latency.

100. Future of LLMs?

Multimodal AI + autonomous agents + personalized AI systems.

☑ How to Use This (Interview Strategy)

👉 Revise daily **20 questions**

👉 Practice explaining verbally (VERY IMPORTANT)

👉 Focus on:

- Transformer
- RAG
- Prompt Engineering
- LLM Deployment

- Agents